Distributing Deep Learning Inference on Edge Devices

ABSTRACT

Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) are widely used in IoT related applications. However, inferencing pre-trained large DNNs and CNNs consumes a significant amount of time, memory and computational resources. This makes it infeasible to use such DNNs/CNNs in resource constrained edge devices, In this research we are trying to implement a distributed inference schema for processing large DNNs and CNNs in such resource constrained edge devices. Our approach of solving this issue is based on partitioning DNNs/CNNs model and distributedly processing the inference tasks using two or more edge devices. Through this poster we introduce our novel approach of distributing the inference process among multiple edge devices while minimizing the network overheads due to communication among devices. In addition to that, we present a task sharing mechanism among working devices and idle devices in the network and a way to convert pre trained models to distributedly executable model format.

KEYWORDS

Distributed Computing, Edge Computing, Distributed Inference, Internet of Things.

MOTIVATION and BACKGROUND

With the rise of the Internet of Things era more and more deep learning applications are used in many IoT applications. Deep neural networks and Convolutional neural networks can be trained on resourceful computers and servers. Such pre-trained DNN/CNN models can be fed previously untrained input data and get results with identified or classified patterns in the input data. This phase is called inference. Even Though it does not require huge computational power compared to training, executing inference tasks in resource constrained edge devices is still a challenging task [1]. IoT devices have been facing the issue of low resources (e.g. computational capacity, memory, energy) since the beginning of the IoT era [2]. Due to the lack of resources, most DNN/CNN models present intolerable time delays in inference execution. Therefore, our motivation is to find a way to reduce this time delay. We aim to do this by distributing the inferencing workload over multiple nearby IoT devices.

Two main facts to be considered while improving time consumption is accuracy of the results and the user data privacy. As we assume all IoT devices in the network are owned by the user, privacy is preserved compared to cloud based processing or processing at an edge server. Also, we assume that connectivity or availability issues will not be experienced as processing happens on a set of local devices that are well-connected to each other. We do not use techniques like model pruning to reduce the model size, as it may affect the accuracy of results.

We need to consider the communication overhead, incurred while distributing the inference process. The inference time reduction should be higher than the communication overhead between end devices in order for the process to be beneficial.

METHODOLOGY and RESULTS

The high level system architecture of the proposed system is shown in Figure 1. There are 2 main components in our solution. Those are a pre-trained Model Extraction Unit (MEU), and an inference Engine which contains a work partitioner, task distributor and communication handler.

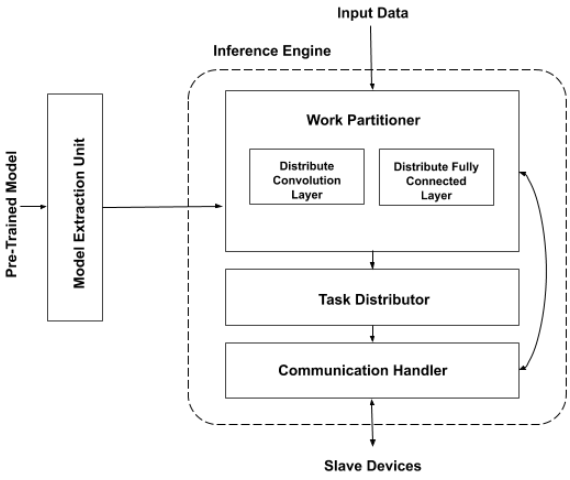

Figure 1 : High Level System Architecture

Model Extraction Unit (MEU) reads and extract data from pre-trained models (e.g. TensorFlow, Yolo). Which reads those models and builds a JSON array for model architecture. And MEU loads the weight data array to the device memory. Inference Engine distributes the input data and parrallary process parts of the model in slave devices which are connected through the network configurations. Upon receiving results from slave devices, the main device will generate the final results of each step.

Most CNNs contain both Convolutional layers and fully connected layers. When it comes to the distribution of the inference process of a CNN, we have to consider the different layer types and different operations happening inside the neural network model. We noticed that the processing happening at convolution layers takes more time than the fully connected layers. Therefore, our main focus was to reduce the time consumption at the convolution layers. We are testing different partitioning methods, which can be used to distribute the inference process among the end devices, as explained further below.

Our tool will detect the number of devices available in the local network, and decide in which way the system should partition the inference process in order to get the optimum time reduction.

1. **Distributing Convolution Layers**

Typical CNN network contains various types of layers. These layers can be convolutional layers which executes computationally expensive convolution operations, pooling layers, upsampling layers , fully connected layers, etc. We noticed that a substantial percentage of the total execution time of a CNN is spent on processing convolutional layers. Furthermore, the execution time increases with the dimensions of the input data for each layer.

Followings are a few of proposed partitioning methods for convolutional layers of a model.

1.1 **Input Wise Partitioning (IWP)**

In this partitioning method we divide the input data frames depending on the kernel dimensions and number of available devices in the network. Thus reducing the dimensions of input data for convolutional operations and then do the convolutional operations separately for partitioned data frames in multiple devices. After completion, the convoluted data is recombined.

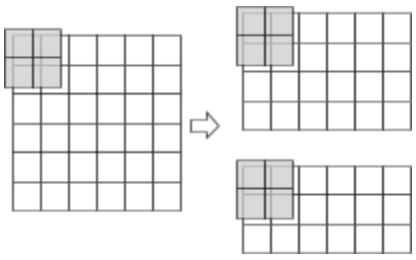

Figure 2 : Partitioning Convolutional Layer Data Frame

Furthermore, we need to consider the communication delay. If the input image is too small, doing the convolution on the main device is more effective compared to the distributed execution.

The performance of the above partitioning schema was evaluated by using a pair of devices and the results are shown in Figure 3. Distributed system gives considerable time reductions compared to single device inference as the input data dimensions increases.

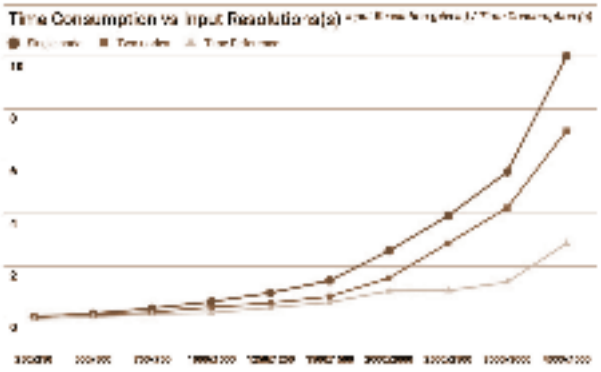

Figure 3 : Comparison of IWP on Single device vs Two Devices

1.2 **Kernel Wise Partitioning (KWP)**

For each convolution layer there may have several feature kernel metrics. Higher number of kernels can affect the convolutional operation execution time. Here we try to assign a set of kernels for each slave device and evaluate the time reduction. However in this case, we need to send the entire input data to all slave devices which may increase the communication delay.

2. **Distributing Fully Connected Layers**

We are investigating the use of model parallelism based partitioning schema for distributing fully connected layers of a CNN/DNN

.

CONCLUSION

In this project we propose, develop and evaluate a novel approach that can be used to distributedly executable inference task of CNNs in resource constrained edge devices. Our current results are promising. We also hope to test the system while dynamically changing the number of end devices that are available.

REFERENCES

[1] J. Chen and X. Ran, “Deep Learning With Edge Computing: A Review,” Proc. IEEE, vol. 107, no. 8, 2019, doi: 10.1109/JPROC.2019.2921977.

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5,pp. 637–646, 2016.